Backend · Data Engineering · Cloud · AI Engineering

خريطة الخلفية المعرفية الكاملة

1. Database / SQL — لازم تكون قوية جدًا

أنتِ بالفعل داخلة في ده:

- ERD + relationships
- Normalization 1NF → 3NF
- PK / FK / constraints
- SQL CRUD
- JOINs
- Subqueries
- CTEs
- Window Functions
- Transactions
- Indexes
- EXPLAIN / EXPLAIN ANALYZE
- Query optimization
- Pagination
- Isolation levels / ACID

دي من أهم الحاجات عندك لأنها بتخدم Backend + DE حتى AI systems اللي فيها data.

2. Backend fundamentals

ذاكري: FastAPI syntax مش بس.

- HTTP بجد
- methods
- status codes
- headers
- cookies
- caching
- REST API design
- Authentication vs Authorization
- JWT
- OAuth conceptually
- API validation
- Error handling
- Middleware
- Dependency Injection
- Service / Repository patterns
- DTOs / schemas
- Pagination / filtering / sorting

- API versioning
- Idempotency
- Rate limiting
- Background jobs
- Webhooks

والأهم، لازم تكوني فاهمة الـflow ده كويس جدًا:

```
Request → API → Business Logic → Database → Response
```

3. Software Engineering

ده ناس كتير بتقفزه وهي بتتعلم Backend. ركزي على:

- Git / GitHub
- Clean code
- SOLID
- Separation of concerns
- Design patterns بشكل عملي مش حفظ
- Testing
- unit tests
- integration tests
- API tests
- Logging
- Configuration / environment variables
- Debugging
- Documentation

Architecture

تفهمي الفرق بين:

- Monolith
- Modular Monolith
- Microservices

ومتى تستخدم كل واحد.

> بالنسبة لمشروعك الحالي أنا مش عايزك تعملي *Microservices* لمجرد إن شكلها *impressive. Modular* محترم هيكون أفضل للتعلم. *monolith*

4. Linux + Networking + Systems

دي من أكتر الحاجات اللي هتخلي الـBackend/Cloud knowledge عندك أعمق. مش لازم تبقي Network Engineer، لكن لازم تفهمي:

Linux

- Filesystem
- Processes
- Permissions
- Environment variables
- Services
- Logs
- Processes / signals
- SSH
- Basic shell commands

Operating Systems / Concurrency

بما إنك أخدتي Advanced Operating Systems، هنا تربطي اللي درستيه بالـBackend والـproduction systems:

- Process vs Thread
- Process lifecycle
- Thread lifecycle
- Context switching
- CPU scheduling conceptually
- Shared memory
- Race conditions
- Synchronization
- Locks / Mutex
- Semaphores
- Deadlocks
- Thread safety
- Concurrency vs Parallelism
- I/O-bound vs CPU-bound workloads
- Async programming
- async / await
- Event loop
- Blocking vs non-blocking operations
- Multithreading
- Multiprocessing
- Worker processes
- Thread pools / process pools

والأهم تفهمي إمتى تختاري كل approach:

- Async ← مناسب جدًا للـI/O-bound operations مثل database/network/API calls.
- Threads ← ممكن تستخدم في بعض الـI/O-bound أو blocking workloads.
- Multiprocessing ← مفيد للـCPU-bound workloads والعزل بين processes.
- Queues + Workers ← للـbackground processing والمهام الثقيلة اللي مش لازم تعطل الـHTTP request.

وده هيربط الـOperating Systems بالـBackend بشكل عملي بدل ما يفضل مجرد مادة نظرية.

Networking

- IP
- Ports
- TCP/UDP
- DNS
- HTTP/HTTPS
- TLS concept
- Reverse proxy
- Load balancer
- Firewall
- Localhost
- Public/private IP
- NAT

لازم تكوني فاهمة رحلة الـrequest من الـclient للـserver والـdatabase، مش مجرد حفظ المصطلحات.

Docker + CI/CD .5

أنتِ أصلًا ناوية تتعلميهم، وده ممتاز. ذاكري:

- Docker image
- Container
- Dockerfile
- Volumes
- Networks
- Docker Compose
- Environment variables
- Container-to-container communication
- Health checks

وبعدها افهمي الـCI/CD pipeline:

```
Git Push → CI → Tests → Build Docker Image → Deploy
```

وتفهمي:

- CI vs CD
- GitHub Actions
- Secrets
- Build/test/deploy pipeline

6. Cloud

هنا متتعلميش AWS + Azure + GCP مع بعض. اختاري واحدة كبداية.

مثلاً AWS:

- EC2
- S3
- RDS
- IAM
- VPC concept
- Security Groups
- Load Balancer
- CloudWatch
- Basic networking

مش مطلوب تحفظي 50 AWS service. الهدف إنك تعرفي:

> "لو عندي *FastAPI + PostgreSQL*، إزاي أحطهم *online* بشكل محترم؟"

وده أهم بكتير من حفظ services.

7. Data Engineering

هنا عندك roadmap كويسة جدًا بالفعل:

- SQL
- Data Modeling
- ETL / ELT
- Python data processing
- Airflow
- Data Quality
- Warehouse
- Spark
- Kafka

بس فيه حاجتين أضيفهم:

Data Warehousing

- OLTP vs OLAP
- Fact tables
- Dimension tables
- Star schema
- Snowflake schema
- ETL vs ELT

Data formats

- CSV
- JSON
- Parquet
- Avro conceptually

خصوصًا Parquet لأنه مهم جدًا في الـDE.

8. Distributed Systems — مش بعمق، لكن foundation

دي مهمة جدًا ليكي لأنك عايزة Backend + Cloud + DE + AI. افهمي concepts زي:

- Scalability
- Horizontal vs vertical scaling
- Stateless services
- Caching
- Redis
- Queues
- Asynchronous processing
- Message brokers
- Eventual consistency
- Replication
- Partitioning
- Sharding
- CAP theorem conceptually

وممكن بعدين: Redis، Kafka / RabbitMQ — لكن مش لازم تدخلي Kafka دلوقتي.

LLM Engineering / Product AI .9

أنتِ بالفعل فاهمة الـAI/ML fundamentals، فمش محتاجة نعيد: train/validation/test، overfitting، basic ML metrics، basic feature engineering، basic embeddings concepts.

بدل ده، تركيزك هنا يكون على Advanced LLM Engineering + Product AI.

LLM Engineering

- Advanced prompt engineering
- Prompt templates and composition
- Structured outputs
- Tool calling / function calling
- Model selection
- Model routing
- Fallback models
- Streaming
- Batching
- Concurrency for LLM calls
- Context management
- Context compression
- Long-context strategies
- Token/cost optimization
- Latency optimization
- Model limitations and failure modes
- LLM application architecture

RAG — Advanced

- Advanced chunking strategies
- Semantic chunking
- Hierarchical retrieval
- Metadata filtering
- Hybrid search
- Dense + sparse retrieval
- Reranking
- Query rewriting
- Multi-query retrieval
- Contextual retrieval
- Parent-document retrieval
- Retrieval routing
- Contextual compression
- Retrieval evaluation
- RAG pipelines
- Advanced RAG architectures
- Handling noisy / conflicting documents
- Grounding and citation strategies
- RAG observability

LangGraph / Agentic AI

- Advanced StateGraph design
- Complex state management
- Reducers
- Conditional routing

- Dynamic workflows
- Tool orchestration
- Agentic workflows
- Multi-agent architectures
- Supervisor patterns
- Handoffs
- Subgraphs
- Persistence
- Checkpoints
- Memory
- Human-in-the-loop
- Interrupts
- Retries
- Fallbacks
- Error recovery
- Parallel execution
- Streaming
- Long-running agents
- Agent evaluation
- Agent observability
- Tracing
- Production agent architecture

Guardrails / AI Safety for Applications

- Prompt injection
- Jailbreak awareness
- Input validation
- Output validation
- Structured output enforcement
- PII detection / handling
- Tool permission boundaries
- Tool access control
- Content filtering
- Hallucination mitigation
- Grounding
- Guardrails around agent actions
- Preventing unsafe / unintended tool execution
- Model output verification

LLM Evaluation

- Evaluation datasets
- Golden datasets
- Automated evaluation
- LLM-as-a-judge
- Answer correctness
- Faithfulness / groundedness
- Relevance
- Retrieval quality
- Tool-call accuracy
- Agent trajectory evaluation
- Regression testing
- Offline evaluation

- Online evaluation
- Latency measurement
- Token / cost measurement

Product AI

- Turning an AI capability into a real product feature
- AI feature requirements
- AI UX considerations
- Human-in-the-loop design
- Confidence / uncertainty handling
- Fallback behavior
- AI failure handling
- Feedback loops
- User feedback collection
- AI feature evaluation
- AI product metrics
- Latency vs quality trade-offs
- Cost vs quality trade-offs
- Model selection based on product requirements
- Designing reliable AI workflows instead of simple chatbot demos

Production AI / LLMOps

- Model serving
- Inference
- Latency
- Cost optimization
- Caching
- Rate limits
- Observability
- Tracing
- Evaluation
- Model/version management
- Prompt/version management
- Fallback models
- Retries
- Streaming
- Monitoring
- AI application logging
- Token usage tracking
- Model performance monitoring
- Production debugging
- AI system reliability

وده بالذات هيربط الـAI بالـBackend والـCloud.

وفيه 3 حاجات أنا شايفهم مهمين جدًا ليكي

Caching ①

اتعلمي Redis على مستوى عملي. وتفهمي:

- Cache-aside
- TTL

- Cache invalidation
- Why caching helps
- متى caching يسبب مشاكل

وافهمي كمان استخدام caching في:

- Backend APIs
- Database queries
- Sessions
- LLM responses
- Embeddings / retrieval
- Expensive computations

Async + Queues ②

دي مهمة جدًا للـBackend والـDE والـAI. افهمي الفرق بين الـsynchronous processing والـasynchronous processing، ومتى تحتاجي إن الـrequest ترجع بسرعة وتسيبي processing تقيل يشتغل في background.

وده هيفتحلك:

- Celery
- Redis
- RabbitMQ
- Kafka
- Workers
- Background tasks
- Task queues
- Retry mechanisms
- Scheduled jobs

لكن مش لازم تتعلميهم كلهم مرة واحدة.

Observability ③

دي underrated جدًا. افهمي: Logs، Metrics، Traces، ومفاهيم: latency، throughput، error rate، request tracing، health checks.

وفي الـAI تحديدًا:

- LLM latency
- Token usage
- Model cost
- Tool-call traces
- Agent traces
- Retrieval traces
- Evaluation results
- Failure analysis

وده هيبقى مهم جدًا لما تعملي:

```
FastAPI + PostgreSQL + LangGraph + LLM
```

وتحاولي تعرفي فين المشكلة لما السيستم بقى بطيء أو بيضرب errors.